

Step 3 is to add a line near the bottom of the `while` loop to call the `draw()` method of the widget, to make it appear in the window:

```
oWidget.draw()
```

Since we specified the window to draw into, the location, and any details that affect the appearance of the widget in step 1, we don't pass anything in the call to `draw()`.

Adding an Image

Our first example will be the simplest widget: we'll use the `Image` class to display an image in a window. When you instantiate an `Image` object, the only required arguments are the window, the location in the window to draw the image, and the path to the image file. Create the `Image` object before the main loop starts, like so:

```
oImage = pygwidgets.Image(window, (100, 200), 'images/SomeImage.png')
```

The path used here assumes that the project folder containing the main program also contains a folder named *images*, inside which is the *SomeImage.png* file. Then, in the main loop you just need to call the object's `draw()` method:

```
oImage.draw()
```

The `draw()` method of the `Image` class contains a call to `blit()` to actually draw the image, so you never need to call `blit()` directly. To move the image, you can call its `setLoc()` method (short for set location), specifying the new x- and y-coordinates as a tuple:

```
oImage.setLoc((newX, newY))
```

The next time the image is drawn, it will show up at the new coordinates. The documentation lists many additional methods that you can call to flip, rotate, scale, get the image's location and rectangle, and so on.